SWE5006 PRACTICE PROJECT · TEAM 8

BLOCBUY

ONE FOR ALL • ALL FOR ONE

A community group-buying platform with pay-first teams

Two prices • pay first to team up • refunded if the team doesn't fill

1 Introduction · 2 Analysis, design, patterns · 3 Testing · 4 CI/CD demo · 5 System demo

Lim Jiechao · Liu Jin · Panpakorn Siripanich · Wang Ziang · Yu Qianshuo

Graduate Certificate in Designing Modern Software Systems · 26 October 2026

Group buying fails when buyers cannot commit together

> **Small merchants can discount only against committed volume, and no single buyer can reach it: demand exists, but nobody commits first.**

Today

- Merchants discount only when volume is predictable
- One buyer can never reach a bulk price
- Advertising is costly and not tied to real purchases

BlocBuy: pay-first teams

- Two prices: individual and team
- Pay upfront to start a team, then share the invite link
- Friends join by paying upfront too
- Fills in time: team price. Expires: everyone refunded automatically

Stripe test mode only (proposal §6.1)

Scope: nine use cases, six quality attributes with targets

IN SCOPE · PROPOSAL §6.1

UC1 Browse listings, two prices
UC2 Sign in with Google
UC3 Start a team, pay first
UC4 Join a team, pay first
UC5 Team succeeds or expires
UC6 Refund on expiry
UC7 Share an invite, credit the sharer
UC8 My orders and my teams
UC9 Admin: listings and team terms

+ Receive notifications (Sprint 4, SWE5001; off on dev)

OUT OF SCOPE · PROPOSAL §6.2

Merchant self-service · analytics · recommendations · automatic team matching · email, SMS, push · native apps · live payments

QUALITY ATTRIBUTES AND TARGETS · PROPOSAL §6.3

Attribute	Target	Shown in
Correctness under concurrency	team capacity never exceeded; no payment double-charged or double-refunded	**§5 demo · §3**
Performance	interactive operations ≤ 2 s at 100 concurrent users; expiry resolved ≤ 1 min after the window	**§5 k6 run · §3**
Security	Google sign-in; role-based admin; signed Stripe webhooks; no credentials in code	**§5 demo · §4**
Maintainability, extensibility	new pricing, payment provider or team consequence without editing domain logic	§2 patterns · §4
Code quality	SonarCloud gate on every PR; unit tests on the complex classes	§3 · §4
Auditability	every payment, refund and transition traceable; every artefact traceable to a commit	§4

Promised, delivered, changed, dropped: status on 21 October

Promised (proposal)	Status on 21 Oct	Record
UC1–UC9	DELIVERED admin console beyond scope	#24–#32 · F-ADR 0015
Pay-first START, JOIN, refund on dev	OPEN Checkout returns; webhook never confirms	#517 · #223
State for the team lifecycle	CHANGED transition table; State on listings	design doc · slide 21
Google sign-in with JWT	CHANGED Sa-Token session, HttpOnly cookie	D6/D7 · F-ADR 0002
Spring Data JPA	CHANGED MyBatis behind repository ports	skeleton plan §3
Dev on ECS Fargate and RDS	CHANGED one EC2 host, Compose, DB on host	ADR 0008
Secrets in Secrets Manager	CHANGED SSM SecureString, root-only env file	operator guide §7
Human-gated prod, ECS circuit breaker	DROPPED for SWE5006; built for SWE5001	#226 · ADR 0008
E2E suite, auto burndown, saved-plan Terraform	PARTIAL stubs; daily data, no chart; manual	F-ADR 0011 · #229

Outstanding #517 and #223 (go/no-go 23 Oct) · SSH 22 open on the dev host · CodeQL off · CloudFront → ALB over HTTP until 16 Nov (#227) · dev volume unencrypted

Milestones achieved and effort per member

MILESTONES ACHIEVED

Date	Milestone
27 Aug	Proposal submitted
5 Sep	Backend pipeline live on dev
14 Sep	Sprint 1 tag
20 Sep	Stripe Sandbox acceptance (local)
22 Sep	Web app live on dev
27 Sep	Sprint 2 tag
30 Sep	Performance baseline (local)
11 Oct	Sprint 3 tag
13 Oct	Admission refactor (PR #424)
23 Oct	Sprint 4 tag `[FILL: date]`
—	Paid start, join, refund on dev: not yet

EFFORT PER MEMBER · PLANNED HOURS `[VERIFY: S4 split, PR #538]`

Member	S1	S2	S3	S4	Planned	Actual
Liu Jin	—	30	30	51	**111**	`[CONFIRM: agreed]`
Yu Qianshuo	—	30	30	48	**108**	`[CONFIRM: agreed]`
Wang Ziang	—	30	30	46	**106**	`[CONFIRM: agreed]`
Panpakorn	—	30	30	41.5	**101.5**	`[CONFIRM: agreed]`
Lim Jiechao	—	30	30	38	**98**	`[CONFIRM: agreed]`

Sprint 1 hours were not recorded. Team done: S2 134 of 150 h, S3 117 of 150 h. Budget in the proposal: 10 man-days each (§7).

Planned, not actual: no member logged actual hours, and the progress reports carry none. `[CONFIRM: actual effort agreed by all five]`

Ownership: one use case and one design problem each

Member	Use case	Design problem → GoF pattern	Slide	Change from proposal §6.1
Liu Jin	UC9 admin listings and team terms	Listing lifecycle → **State** (full form)	20–21	was UC5 State, UC9
Wang Ziang (Eric)	UC3 start, UC4 join	Team creation → **Factory Method**	16	unchanged; UC4 race is a transition strategy
Yu Qianshuo (Yorik)	UC6 refund on expiry	Team outcome consequences → **Observer**	17	was Adapter
Panpakorn (Pan)	Receive notifications	Outbox transports → **Template Method**	18	was UC1 Strategy, UC2
Lim Jiechao	UC8 my orders and teams	Pricing → **Strategy**	19	was UC7 Observer, UC8

Each member presents their own design problem in section 2. The team-formation statechart (UC4–UC6) is Eric's, with his UC3/UC4 design.

Report: use case in §3.4, design problem in §3.5 `[FILL: section numbers (#463)]` `[VERIFY: UC9 owner]`

Four sprints: the Definition of Done reached, and what each retro changed

DoD ladder (EPAT 01 Agile p26) coded → tested → integrated → releasable → released

Sprint	Goal	Definition of Done reached
1 · 2–12 Sep	Walking skeleton: sign-in, browse, admin on dev	**integrated**: backend on dev 5 Sep; no frontend yet
2 · 14–25 Sep	Pay, join a team, see the order, replay-safe	**tested** for the money path (Checkout off on dev); 134 of 150 h
3 · 28 Sep–9 Oct	Invite → pay → refund on deployed dev	**integrated**; goal not met under our "on dev" rule; 117 of 150 h
4 · 12–23 Oct	Complete system on dev; three services on ECS (SWE5001)	`[FILL: step as of the 25 Oct rehearsal]`

Retro improvement → change in the next sprint

1 → 2	No burndown, no hours	→ daily board snapshots, dated day logs
2 → 3	A 10,000-line PR; cliff burndown	→ PRs under 1,500 lines, mid-sprint check, "on dev" rule in the DoD
3 → 4	`[FILL: retro #451]`	→ owners keep board estimates current, checked mid-sprint

Agile artefacts: a story card, a burndown, a practice

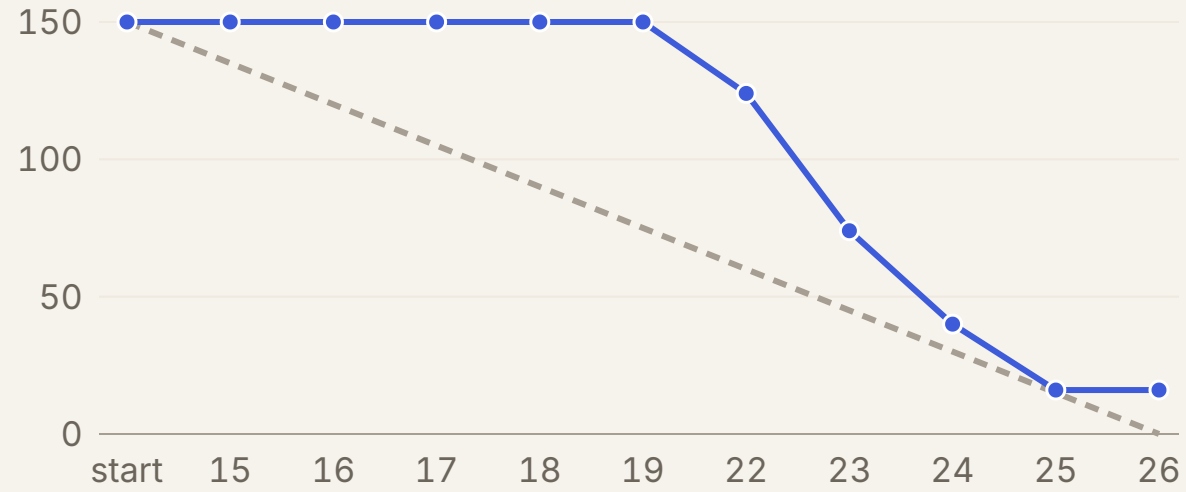
STORY CARD · #221 (US25) — Liu Jin · PR #242

As an admin, I can set a listing's team size, lifetime and team price, so that buyers can start blocs on real listings.

- [x] Admin-only API; 403 non-admin, 401 anonymous
- [x] Size ≥ 2, lifetime in range, team price below individual; 400 with field errors
- [x] Stale expectedVersion → 412; writes audited
- [x] Configured listing is startable; START integration test
- [x] Contract changes additive, merged before the controller
- [x] Unit, PostgreSQL IT, authorization tests; ArchUnit green

Design problem field: State, Optimistic Offline Lock

SPRINT 2 BURNDOWN · REMAINING HOURS, 15–26 SEP



Flat for a week, then a cliff: work merged late in large PRs. Reconstructed from merge dates.

Practice (EPAT 03, 05): two red tests reproduced a quote defect (7 Oct log), then PR #424 refactored admission into shared guards and a Factory Method. Full set of artefacts: backup B1.

WZ Eric

Logical architecture: COLA layers across six bounded contexts

Rubric rows: **2.1**

[FILL: slide 9 draft pending]

WZ Eric

Transition strategies from analysis to design

Rubric rows: **2.2**

`[FILL: slide 10 draft pending]`

WZ Eric

One strategy pair: analysis objects become design classes

Rubric rows: **2.2**

`[FILL: slide 11 draft pending]`

WZ Eric

Use case diagram: in scope, out of scope and status

Rubric rows: **2.3**

`[FILL: slide 12 draft pending]`

WZ Eric

UC4 join: the analysis model

Rubric rows: **2.4**

`[FILL: slide 13 draft pending]`

WZ Eric

UC4 join: the design model

Rubric rows: **2.4**

[FILL: slide 14 draft pending]

WZ Eric

Team-formation statechart

Rubric rows: **2.4**

`[FILL: slide 15 draft pending]`

UC3 start a team: Factory Method

Rubric rows: **2.5**

[FILL: slide 16 draft pending]

YQ Yorik

UC5 to UC6 team outcome: Observer

Rubric rows: **2.5**

[FILL: slide 17 draft pending]

Notifications: Template Method for outbox transports

Rubric rows: **2.5**

[FILL: slide 18 draft pending]

UC1 pricing: Strategy

Rubric rows: **2.5**

[FILL: slide 19 draft pending]

LJ Liu Jin

Design problem: listing rules that depend on its state

Rubric rows: **2.5**

[FILL: slide 20 draft pending]

LJ Liu Jin

After: the State pattern, and why the team uses a table

Rubric rows: **2.5**

`[FILL: slide 21 draft pending]`

LJ Liu Jin

Database schema: core entities and mapping decisions

Rubric rows: **2.6**

`[FILL: slide 22 draft pending]`

LJ Liu Jin

Testing on the Agile Testing Quadrants

Rubric rows: **3.1, 3.3**

`[FILL: slide 23 draft pending]`

LJ Liu Jin

Scripted performance test: tool, scenario, target

Rubric rows: **3.2**

[FILL: slide 24 draft pending]

PS Pan

Source code and artefacts

Rubric rows: **4.2**

[FILL: slide 25 draft pending]

PS Pan

Continuous integration: pull-request jobs and post-merge jobs

Rubric rows: **4.3**

[FILL: slide 26 draft pending]

PS Pan

Security controls by pipeline stage

Rubric rows: **4.3, 3.3**

[FILL: slide 27 draft pending]

PS Pan

Delivery: environments, approval and verification

Rubric rows: **4.4**

[FILL: slide 28 draft pending]

Technical findings and issues encountered

Rubric rows: **4.5**

[FILL: slide 29 draft pending]

PS Pan

Live: one change from pull request to dev

Rubric rows: **4.1, 4.6**

[FILL: slide 30 draft pending]

YQ Yorik

System demo: one bloc from start to refund

Rubric rows: **5.1, 5.2**

`[FILL: slide 31 draft pending]`

YQ Yorik

Live: two buyers, one bloc

Rubric rows: **5.1, 5.2, 5.5**

`[FILL: slide 32 draft pending]`

YQ Yorik

Performance against the target

Rubric rows: **5.3**

[FILL: slide 33 draft pending]

YQ Yorik

Scalability: a stated trade-off

Rubric rows: **5.2**

[FILL: slide 34 draft pending]

YQ Yorik

Maintainability: the pipeline you just saw

Rubric rows: **5.4**

`[FILL: slide 35 draft pending]`

Thank you

Questions?

github.com/PPsyrius/iss-groupbuy-team8 · d1b3np88z657cp.cloudfront.net

Backup slides: index

B1 **Agile artefacts appendix**
Where are the backlog, stories, sprint plans, reviews and retros?

B2 **Sprint 2 and 3 burndowns**
How did earlier sprints burn down?

B3 **ADR index**
Which decisions are recorded, and where?

B4 **Database schema, all tables**
Which tables does each context own?

B5 **Expiry and refund sequence**
How does an expired team become refunds?

B6 **UC6 refund recovery: candidates and decisions**
How do you never refund twice?

B7 **Test inventory**
How many tests of each kind, and where do they run?

B8 **Workflows, required checks and timings**
What runs when, how long, and how often is it green?

B9 **Known gaps and limitations**
What is not done yet in the pipeline and on dev?

B10 **SWE5001 production topology**
What would scaling out look like?

B11 **Stripe Sandbox refund, local evidence (20 Sep)**
Is the refund real, and where has it been proven?

B12 **Scale-out and load results (SWE5001 ECS + local baseline)**
How does it scale, and how fast is it?

Click a row (or type its number in the URL as #B6). Slide 36 stays up during questions; open a backup only when a question needs it.

JC Lim

Agile artefacts appendix

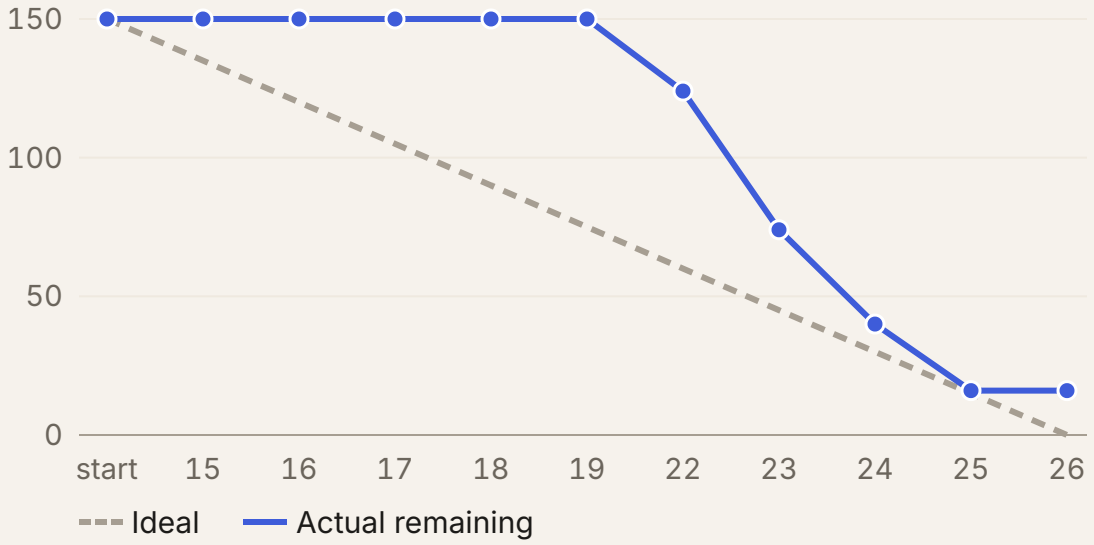
Artefact	Where it lives	State on 21 Oct
Product backlog	GitHub Issues and Project board 2 (GroupBuy Kanban Board); issue templates in .github/ISSUE_TEMPLATE	170 items: 137 Done, 15 Ready, 15 Backlog, 3 In progress
User stories with acceptance criteria	UC issues #24–#32; story issues (e.g. #221, #223, #180); AC per story in docs/agile/sprint-03/plan.md §5	template: complexity, accountable members, design problem, AC
Sprint plans	S1 #49; S2 Sprint 2 plan (docx) and #93; S3 docs/agile/sprint-03/plan.md and #232; S4 docs/agile/sprint-04/plan.md and #465	all four sprints
Burndown charts	S2 chart in docs/sprint2-progress-report.typ; daily data docs/agile/metrics/*.csv (board-snapshot.yml, since 17 Sep)	S2, S3 reconstructed (B2); S4 from snapshots
Sprint reviews	S1, S2 in the progress reports; S3 docs/agile/sprint-03/review.md	S4 due 23 Oct `[FILL: S4 review]`
Retrospectives	S1 report; S2 report §Sprint Retrospective	S3 missing (#451) · S4 due 23 Oct `[FILL: S3, S4 retros]`
Day logs, DoD	docs/agile/sprint-0N/log/ (S2–S4); DoD in the S2 report, sprint-03/plan.md §8, sprint-04/plan.md §8	dated logs per working day

Source: board and issues read with gh on 21 Oct; docs/agile/README.md. The report appendix holds the full set (#463).

Sprint 2 and 3 burndowns

Sprint 2 · remaining hours, 15–26 Sep

150 h planned · 16 h left · reconstructed from merge dates

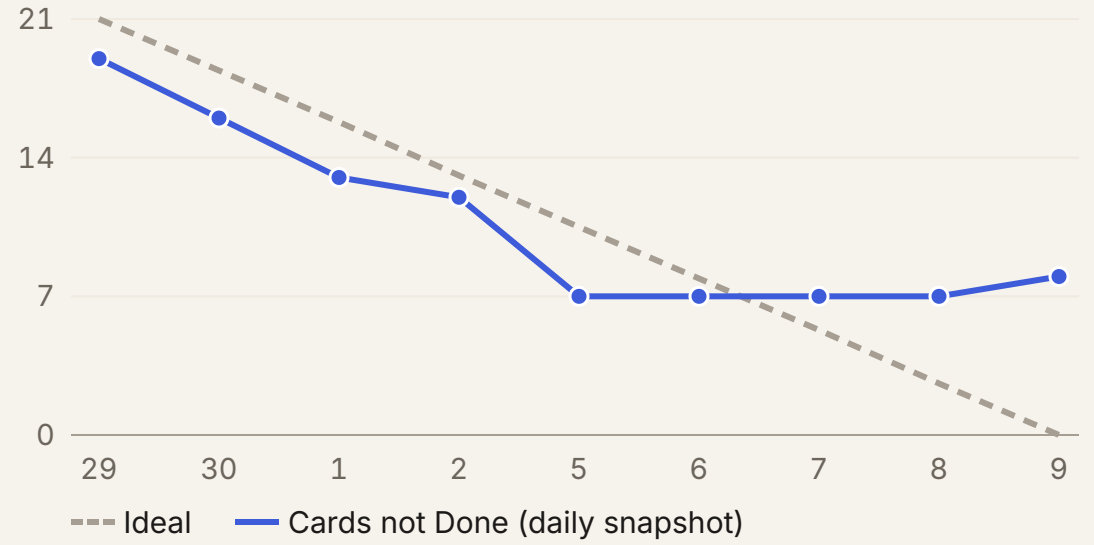


Flat for a week, then a cliff: work merged late in very large pull requests. Retro answer: PRs under 1,500 lines and hours on the board daily.

Source: docs/sprint2-progress-report.typ, "Daily burndown data"

Sprint 3 · open sprint cards, 29 Sep–9 Oct

21 cards · 150 h planned · 33 h left at freeze (review)



Steadier burn, then stuck at 7 from 5 Oct: #223, #224 and #229 waited for Checkout on dev, #226 for the NFR-37 decision. Hours come from the review.

Source: docs/agile/metrics/.csv, sprint-03/review.md*

WZ Eric

ADR index

Backend · backend/docs/adr

#	Decision
0001	Purchase intent lives in order; group terms on the product
0002	Team resolution is published; consequences run before commit
0003	Referral credit is a post-commit subscriber
0004	Retire the unused legacy PaymentIntent path
0005	Gate Stripe Checkout with a deployment property
0006	Payment owns the admin attention read model
0007	Outbox, one SNS topic, one SQS queue per consumer (Proposed)
0008	One codebase, two topologies: dev monolith, three services on ECS

Frontend · frontend/docs/adr

#	Decision
0001	React SPA, pure CSR; Router owns URL state, Query server state
0002	Same-origin /api with an HttpOnly cookie session
0003	Generated API client, not committed
0004	One build artifact promoted dev to prod
0005	SPA fallback by a CloudFront Function
0006	oxlint and oxfmt toolchain
0007	Hand-managed dev CloudFront, then Terraform import
0008	Documentation and evidence conventions
0009	Result envelope: one unwrapResult as select
0010	React Compiler via the Babel preset
0011	Testing strategy: Vitest, MSW, Playwright smoke
0012	Agent configuration in AGENTS.md
0013	pnpm and Node pinned, exact versions
0014	Stripe Hosted Checkout with a server-owned intent
0015	Admin console shell, roles guard, versioned writes

LJ Liu Jin

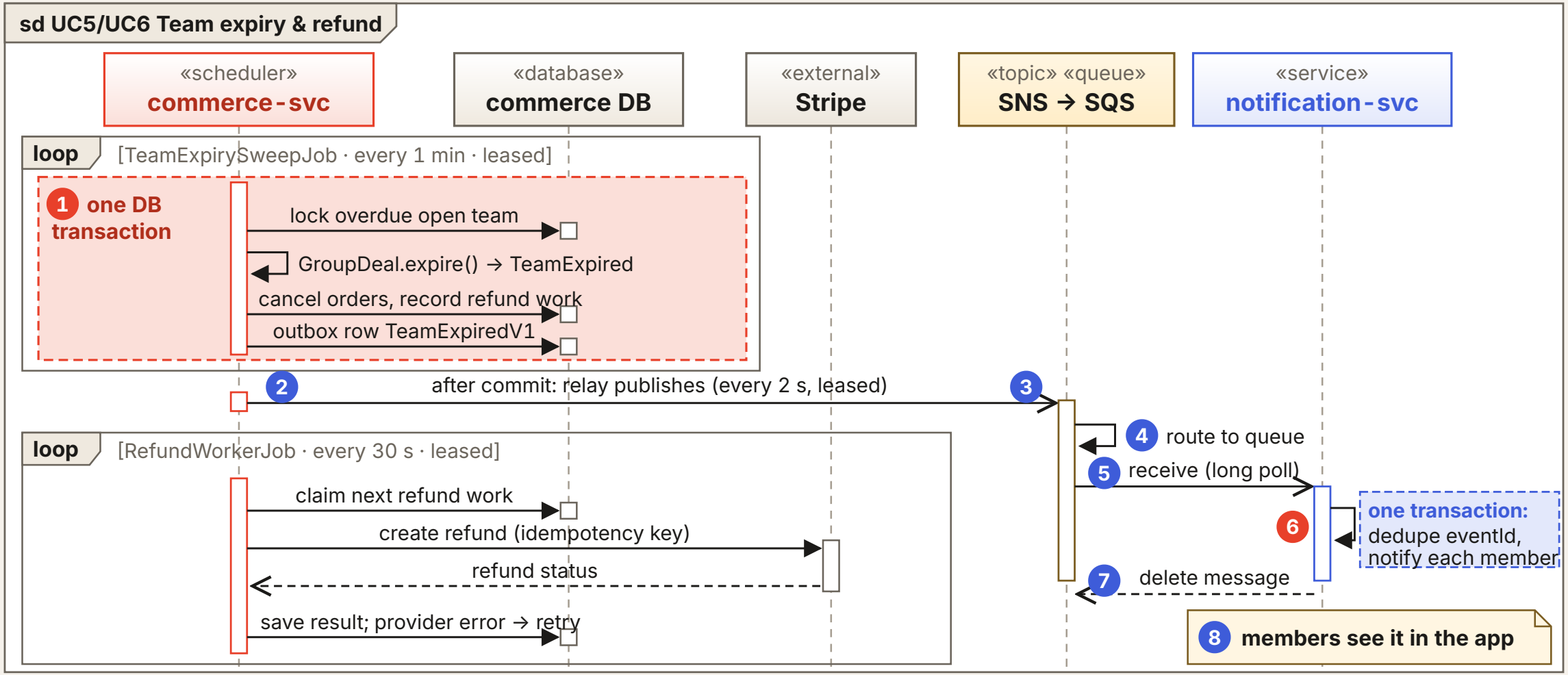
Database schema, all tables

Context	Tables	Migrations	Keys that make “once” hold
identity	`sys_user, sys_user_oauth, sys_role, sys_permission, rel_user_role, rel_role_permission`	V3, V24	one binding per Google subject
catalog	`biz_product, biz_team_price_policy, biz_team_price_quote, log_product_team_terms`	V4, V6, V14, V20, V21	biz_product.version (Optimistic Offline Lock)
groupbuy	`biz_group_deal, biz_group_member, biz_group_participation_receipt, biz_share_invite, biz_share_credit`	V5, V7, V18	one member per user and team; one receipt per payment
order	`biz_order, biz_purchase_intent`	V12, V15, V19, V22	intent fulfilled by one payment
payment	`biz_payment, log_payment_webhook_event, biz_refund, log_payment_review, biz_payment_review_case, log_refund_attempt`	V2, V9–V11, V16	webhook event_id; one refund per payment
notification	`biz_notification, sys_consumed_message`	V27, V28	PK (event_id, consumer); (event_id, user_id)
jobs, outbox	`sys_outbox, sys_scheduler_lock, sys_scheduled_item_retry, sys_checkout_expiry_cursor`	V13, V17, V25, V26	one lease per job; outbox id = eventId

28 Flyway files, 29 tables in schema `domain`; V1 sets the rules: snake_case, timestamptz, numeric money, invariants in the database. On ECS (ADR 0008): one database per service on one RDS instance; identity-svc and notification-svc own their rows above, commerce-svc the rest.

YQ Yorik

Expiry and refund sequence



As built for ECS. The SNS → SQS hop and notification-svc (steps 2–8) are the ECS topology; on dev the monolith runs no broker.

YQ Yorik

UC6 refund recovery: candidates and implementation decisions

Candidate	Decision	Why
Mark refunded, then call Stripe	**Rejected**	A crash would report money returned that never left
Hold a DB transaction across the call	**Rejected**	A rollback cannot undo a refund; locks held over network I/O
Idempotency Key per obligation	**Selected**	Every retry reuses one request identity
Retry after a delay	**Selected**	Fixed 30 s in code, not exponential backoff
Human-in-the-loop review	**Selected**	Mismatched or ambiguous work leaves automatic claims
Distributed transaction with Stripe	**Rejected**	No shared protocol across PostgreSQL and the provider

Implementation decisions

1. Obligation commits with the expiry, before any I/O
2. Key from the obligation ID: `"blocbuy-refund-" + id`
3. Short claim and finish transactions; lease and version fence workers
4. Verify payment facts before, provider snapshot after
5. After 23 h: search Stripe by metadata, never create
6. Reconcile only retrieves an existing refund; no admin endpoint exists

Source: docs/swe5006/report-drafts/yorick-yu-contribution.md. Evidence is local with a provider substitute.

LJ Liu Jin

Test inventory

Kind	Tools	Count by module	Example classes	Runs
Unit	JUnit Jupiter, Mockito	182 *Test classes (by file name; 21 start Spring or a database): domain 42, application 41, infrastructure 47, bootstrap 43, adapter 4, platform 2, commerce-svc 2, client 1	`RefundServiceTest`, `PaidGroupDealTest`, `ProductStatusTest`	every PR (build-quality)
Integration	Testcontainers: PostgreSQL 18, Redis 8.10, ElasticMQ	49 classes: bootstrap 24, infrastructure 15, services 9, platform 1	`OrderRepositoryIT`, `ServiceGrantsIT`, `OutboxRelayElasticMqIT`, `StripeRolloutAcceptanceIT` (acceptance)	every PR (build-quality)
Architecture	ArchUnit	18 rules in 1 class	`ArchitectureTest`	every PR (build-quality)
API contract	springdoc export, git diff, openapi-diff 2.1.7	1 file	`contracts/openapi.json`	backend PRs (contract-check, not required)
Component	Vitest, Testing Library, MSW	76 files: features 58, routes 10, components 3, test 3, api 2 · 799 tests	`account.test.tsx`, `admin-listing-edit.test.tsx`	every PR (frontend-check)
End-to-end	Playwright, Chromium	7 specs, 18 test declarations	`buyer-journey.spec.ts`, `quote.spec.ts`	PR (e2e-preview), after deploy (e2e-dev)
Performance	Node workload; k6 (SWE5001)	3 scenarios; 5 k6 tooling test files	`backend/perf/workload.mjs`, `perf/k6/`	by hand (tooling tested in CI)

Counted 19 Oct at 8bcb38dd: 183 `*Test.java` (182 + ArchitectureTest), 49 `*IT.java` (45 start containers). Last full runs: backend 1,931 (16 Oct log), frontend 799 (19 Oct log).

PS Pan

Workflows, required checks and timings

Workflow · trigger	Jobs (required in blue)	Median	Green
ci-backend · PR	**required**: build-quality (13.1 min), docker (2.8 min), SonarCloud Backend; contract-check (12 s)	16.1 min to docker end (18 runs, 16–19 Oct)	18 of 20 finished runs; 16 cancelled by a newer push
ci-backend · push to dev	package-dev 46 s → publish-ecr-dev 73 s → deploy-existing-ec2 40 s; SonarCloud Backend refreshes the dashboard	3.1 min to deployed (88 runs)	deploy 88 of 97; all 9 failures 14–17 Sep (SSM: instance not in a valid state)
ci-frontend · PR	**required**: frontend-check (3.4 min), SonarCloud Frontend; e2e-preview (69 s), intent-stale	4.8 min to e2e-preview end (19 runs)	19 of 20 finished runs
ci-frontend · push to dev	package-dev 35 s → deploy-s3-dev 21 s → e2e-dev 82 s; SonarCloud Frontend	2.7 min to e2e-dev green (94 runs)	deploy 97 of 97; e2e-dev 94 of 97 (3–4 Oct)
security · PR, dev, weekly	**required**: gitleaks (10 s); trivy-deps (72 s), trivy-config, workflow-lint, deployment-bundle when their paths change	12 s on most PRs	PR 97 of 98 finished; dev push 100 of 100
rollback-backend · manual	validate, then SSM rollback to the recorded previous digest or a given one	—	never run
SWE5001 · prod branch	swe5001-release-gate, swe5001-deploy, infra-swe5001 (promotion PR; see the SWE5001 deck)	—	—

Last 100 runs per workflow and event, read with `gh run list` and the jobs API on 19 Oct. PR durations count only runs where the gated jobs ran (docs-only PRs skip them in about 10 s). Push medians run from the push run start, not the merge click.

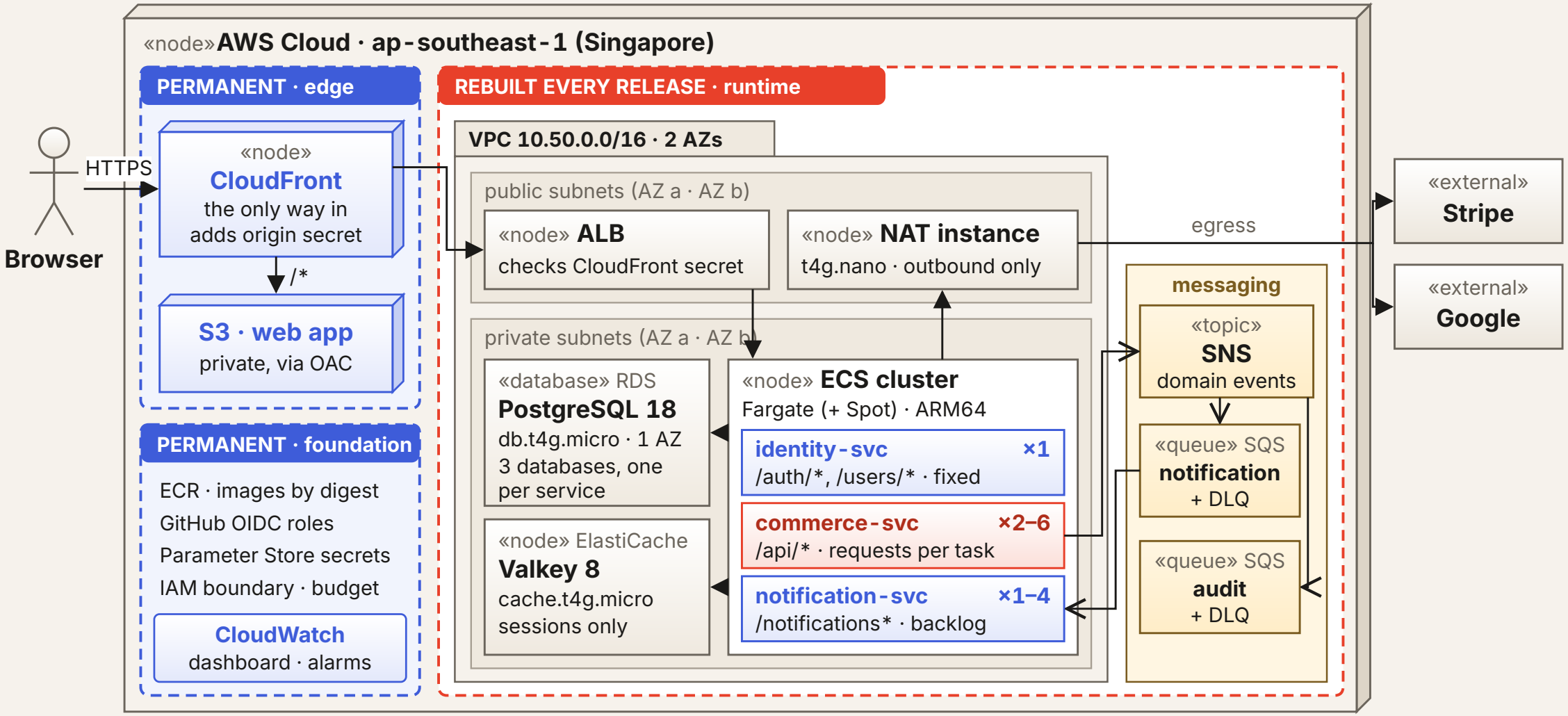
PS Pan

Known gaps and limitations

Gap	Effect	Status
CodeQL off	no CodeQL SAST; SonarCloud and SpotBugs remain	ENABLE_CODEQL=false; code scanning not enabled on the repository
CloudFront → ALB over HTTP	/api payloads and the origin header cross the AWS network unencrypted	NFR-37 TLS still unmet; accepted risk on dev until 16 Nov (#227, NFR register); fix needs an origin hostname and ACM
SSH open to the internet	port 22 from 0.0.0.0/0 on the dev host, which has a public IP; SSM sessions make it unnecessary	AWS read-only check, 19 Oct; open item 16
Dev EBS volume not encrypted	database and host data at rest unencrypted	AWS read-only check, 19 Oct; not raised as an issue yet
Rollback never exercised	recovery path untested on the real host	rollback-backend.yml: 0 runs; no automatic rollback has fired
No dev alarms	failures are noticed by e2e-dev or by people	all six CloudWatch alarms belong to SWE5001
Dev not zero-downtime	a deploy restarts the only API container (502 on 3–4 Oct)	accepted for dev
Secrets in a host file	manual creation and rotation, root-only 0600	accepted for dev; prod uses Parameter Store
Rebuilt after merge	the deployed image and dist/ are not the PR-scanned artefacts; Trivy not re-run	strict up-to-date checks; ECR scans on push; ADR 0004 text to reconcile
Gitleaks scans selected commits	24 historical matches still to decide	docs/quality/quality-gates.md, 14 Oct

PS Pan

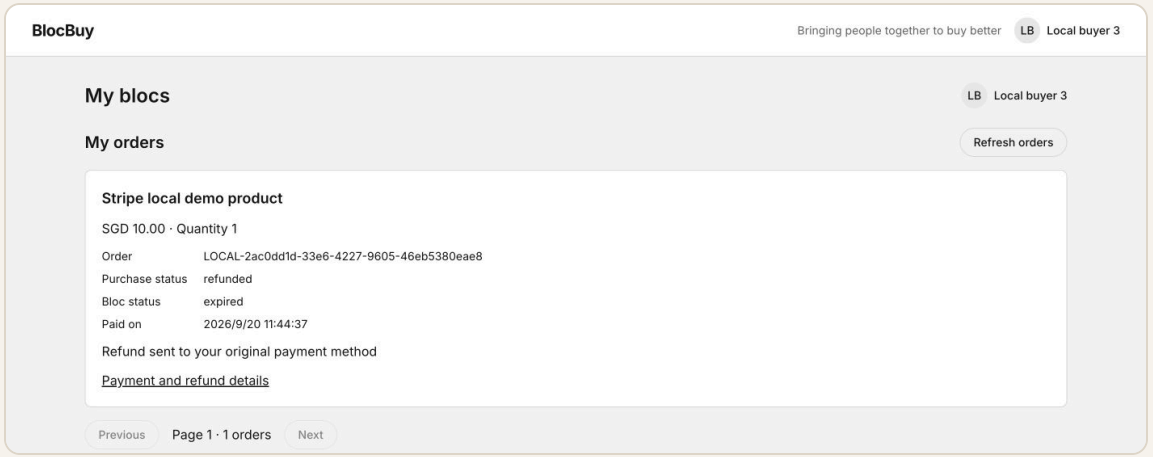
SWE5001 production topology



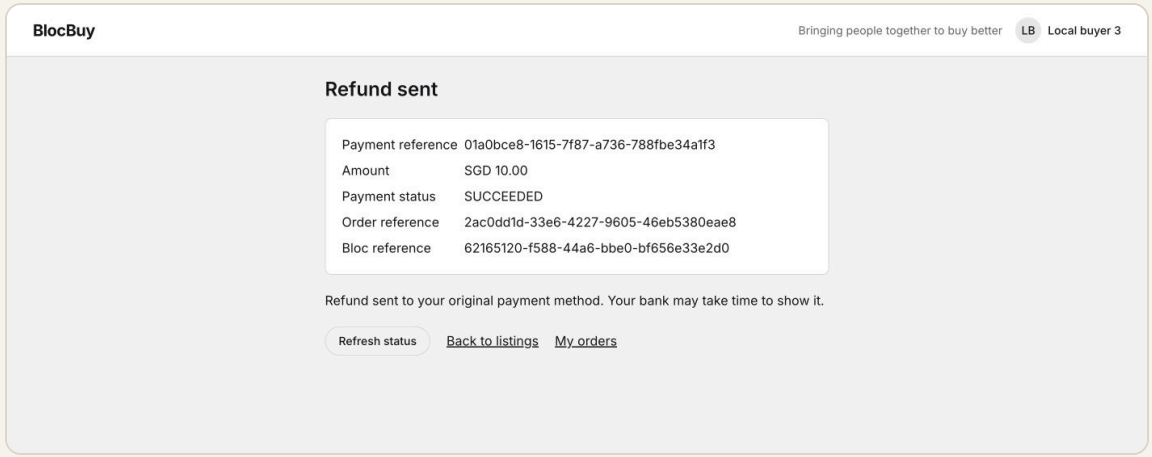
SWE5001 prod: the same code on ECS, rebuilt from scratch on every release and destroyed after it. SWE5006 dev is slide 28.

YQ Yorik

Stripe Sandbox refund, local evidence (20 Sep)



My orders: the expired bloc's order is refunded (UI of 20 Sep, since redesigned)



Receipt: "Refund sent", payment 01a0bce8..., SGD 10.00

Scenario	Result on 20 Sep (Team8 Stripe Sandbox, livemode=false)
START and JOIN	two SGD 10 payments → one member and one order each; the bloc reached succeeded, 2 of 2
Webhook replay	replayed valid events changed nothing; a modified payload with the old signature got 400
Expiry	an under-filled bloc expired → one SGD 10 refund obligation; its unpaid Checkout session expired
Refund	one full Sandbox refund `re_3UHbeV5iEVLN2uQJ1V71krwQ`, succeeded; a recheck created no duplicate

Local, not dev: real Stripe Sandbox calls from a local backend and PostgreSQL; no Google sign-in, no deployed environment. On dev the refund waits for #223 and #517. Source: `backend/docs/payment/stripe-sandbox-acceptance-20260920.md`

YQ Yorik

Scale-out and load results (SWE5001 ECS + local baseline)

SWE5001 PROD · ECS FARGATE

Scale-out

- Target tracking on ALB requests per target: 12,000 a minute, scale-out cooldown 60 s
- commerce 2–6 tasks in two AZs, notification 1–4 (read back 16 Oct)
- V3, 19 Oct: commerce **1 → 3 tasks** in about four minutes, no manual scaling; at 3 tasks ≈ 455 req/s, server p95 13–50 ms, 2 errors in 570,445
- Not V3 acceptance: the workstation generator dropped iterations, so the analyzer rejects the run
- V4 queue drain: `[FILL: backlog peak, notification tasks, drain time (19 Oct)]`

SWE5001 V2 · ECS, 16 OCT

Load, one task

- k6, ten one-minute plateaus from 10 to 200 requests a second, through CloudFront
- 57,600 requests, **0 errors**, run twice
- p95 between 23 and 57 ms on every plateau (target under 500 ms)
- No saturation reached: 200 a second is a tested lower bound, not a limit

DEV CODE · LOCAL RUN, 30 SEP

Workload baseline

100 users each	p95	Errors
Browse (list + detail)	24 ms	0
JOIN intent, unpaid	57 ms	0
My Orders	18 ms	0

30 paid teams expired within 58 s of their deadlines under that load. One laptop with real HTTP, PostgreSQL and Redis; **not a deployed result**.

Dev, where the demo runs: one EC2 host by design (backend ADR 0008), so no auto-scaling there. Same code, two topologies: scale-out is shown on SWE5001 prod.

Sources: docs/swe5001/evidence/20261016-http-baseline-dashboard/README.md (V2, scaling policy); docs/swe5001/evidence/20261019-v3-autoscaling/README.md (V3); docs/perf/sprint3-baseline.md (local workload).